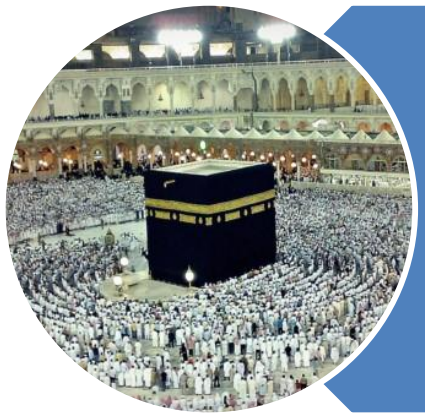
Data Structures - I

CPCS 204

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Course Introduction

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Course Learning Outcomes (CLOs)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Course Learning Outcomes (CLO)

Differentiate between static and dynamic data structures. Determine the tradeoffs between different data structures and identify applications of them.	SO-b
Design and analyze more efficient algorithms for solving basic problems.	SO-j
Comprehend and trace the output of a given piece of code or algorithm.	SO-a
Demonstrate linear and binary search techniques in problem solving.	SO-a
Represent and implement linked data structures.	SO-j
Apply different operations, including search, insertion, and deletion, on linked lists.	SO-a

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Course Learning Outcomes (CLO)

Apply recursion in solving simple problems.	**SO-a**
Implement stacks using arrays and linked lists.	**SO-j**
Implement queues using arrays and linked lists.	**SO-j**
Describe and represent different tree terminologies.	**SO-a**
Apply different operations, including search, insertion, and deletion, on trees and binary search trees.	**SO-k**
Comprehend, compare, and apply sorting algorithms in problem solving.	**SO-a**

CPCS 204

Course Learning Outcomes (CLO)

Evaluate and Analyze the running time of small algorithms in terms of the number of operations performed	SO-b
Experiment and practice recursive sorting algorithms in problem solving	SO-b
Describe, represent, and apply hash table terminologies and operations.	SO-j
Describe, represent, and apply heap (priority queue) terminologies and operations.	SO-b
Develop small-scaled programming assignments in Java, taking space and time efficiency into account.	SO-b

CPCS 204

Goals
Data Structure - I

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Goals

- Improve knowledge of standard data structures and abstract data types
- Improve knowledge of standard algorithms used to solve several classical problems
- Cover some mathematical concepts that are useful for the analysis of algorithms
- Analyze the efficiency of solutions to problems
- Learn about and be comfortable with recursion

Teaching Method

- This class is NOT used to teach you JAVA
- Majority of this class is used covering new data structures, abstract data types, and algorithm analysis
- The teaching of these concepts dictate more explanation and less of a focus on "code"
  - Some code will be shown on the PowerPoint slides
    - Such as after we explain a new abstract data type
    - We'll show the code of how you would implement it
  - **However, writing of actual code will most likely never be done in class**
    - Again, that is not the purpose of this class

CPCS 204

Motivation
Data Structure - I

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Pre-requisites & DS

- In CPCS-202 and 203, we only cared if we found a solution to the problem at hand
  - Didn't really pay attention to the efficiency of the answer
- For this class:
  - We learn standard ways to solve problems
  - And how to analyze the efficiency of those solutions
  - Finally, we simply expand upon our knowledge of our use of the JAVA programming language

CPCS 204

Example Problem

- We will now go over two solutions to a problem
  - The first is a straightforward solution that a CPCS-203 student should be able to come up with
    - **Doesn't care about efficiency**
  - The second solution is one that a CPCS-204 student should be able to come up with after some thought
    - **Cares about efficiency**
- Hopefully this **example** will **illustrate** part of the **goal** of this course

Max Number of 1's

- You are given an n x n integer array
  - Say, for example, a 100x100 sized array
- Each row is filled with several 1's followed by all 0's
  - Example:
    - Row 1 may have 38 1's followed by 62 0's
    - Row 2 may have 73 1's followed by 27 0's
    - Row 3 may have 12 1's followed by 82 0's
    - You get the idea
- The goal of the problem is to identify the row that has the maximum number of 1's.

Max Number of 1's (Basic Solution)

- Straightforward CPCS-203 style solution:
  - Make a variable called MaxOnes and set equal to 0
  - For each row do the following:
    - Start from the beginning of the row on the left side
    - Scan left to right, counting the number of 1's until the first zero is encountered
    - If the number of 1's is greater than the value stored in MaxOnes, update MaxOnes with the number of 1's seen on this row
- Clearly, this works
- But let's see how long this algorithm will take

Max Number of 1's (Example 01)

1	1	0	0	0	0
0	0	0	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	1	1	0
1	1	1	1	0	0

Max Number of 1's (Example 01)

1	1	0	0	0	0
0	0	0	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	1	1	0
1	1	1	1	0	0

- Rows 6 & Columns 6
- # of Comparison 23
- Max. Number of comparisons 6 x 6 = 36

Max Number of 1's (Example 02)

0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0

- Rows 7 & Columns 8
- # of comparisons 30
- Max. Number of comparisons 7 x 8 = 56

Max Number of 1's (Analysis)

- Analysis of Straightforward Solution:
  - Basically we iterate through each square that contains a 1, as well as the first 0 in each row
  - If all cells were 0, we would only "visit" one cell per row, resulting in n visited cells
  - However, if all cells were 1's, we would "visit" all of the cells (n^2 total)
    - So in the worst case, the number of simple steps the algorithm takes would be approximately n^2
  - This makes the running time of this algorithm $O(n^2)$
    - The meaning of this Big-O will be discussed later in the semester

Max Number of 1's (Optimal Solution)

- More Efficient CPCS-204 style algorithm:

1. Initialize the current row and current column to 0
2. While the current row is less than n (or before the last row)
   a. While the cell at the current row and column is 1
      - Increment the current column
   b. Increment the current row
3. The current column index represents the maximum number of 1's seen
4. Now let's trace through a couple of examples

CPCS 204

Max Number of 1's (Example 01)

	0	1	2	3	4	5
0	1	1	0	0	0	0
1	0	0	0	0	0	0
2	1	1	1	0	0	0
3	1	1	1	0	0	0
4	1	1	1	1	1	0
5	1	1	1	1	0	0

- Rows 6 & Columns 6
- # of comparisons 11 (previously 23)
- Max. Number of comparisons 6 + 6 = ~~12~~ 11

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Max Number of 1's (Example 02)

0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0

- Rows 7 & Columns 8
- # of comparisons 14 (previously 30)
- Max. Number of comparisons 7 + 8 = 15

Max Number of 1's (Analysis)

- Analysis of Better Solution:
  - How many steps will this algorithm take, in terms of n?
    - Each "step" taken by the algorithm either goes to the right or down in the table.
    - There are a maximum of n-1 steps to the right
    - And a maximum of n-1 steps down that could be taken
    - Thus the maximum number of "steps" that can be done during this algorithm is approximately **2n – 1**
      - And this is the worst case
    - So the running time of this algorithm is **O(n)**
      - An improvement of the previous algorithm
      - Input size of 100 for n
      - n^2 would be 10,000 steps and 2n would be 200 steps

Implementing an Algorithm in JAVA

- In this class, you will have an opportunity to improve upon your ability to write programs that implement an algorithm you have learned
- You must know the syntax of JAVA in order to properly and effective do this
- There's no set way to create code to implement an algorithm
- We have many set algorithms and data structures that you will study
- Mostly, however, you will simply have to apply the data structures and algorithms shown in class fairly directly to solve the given problems

CPCS 204

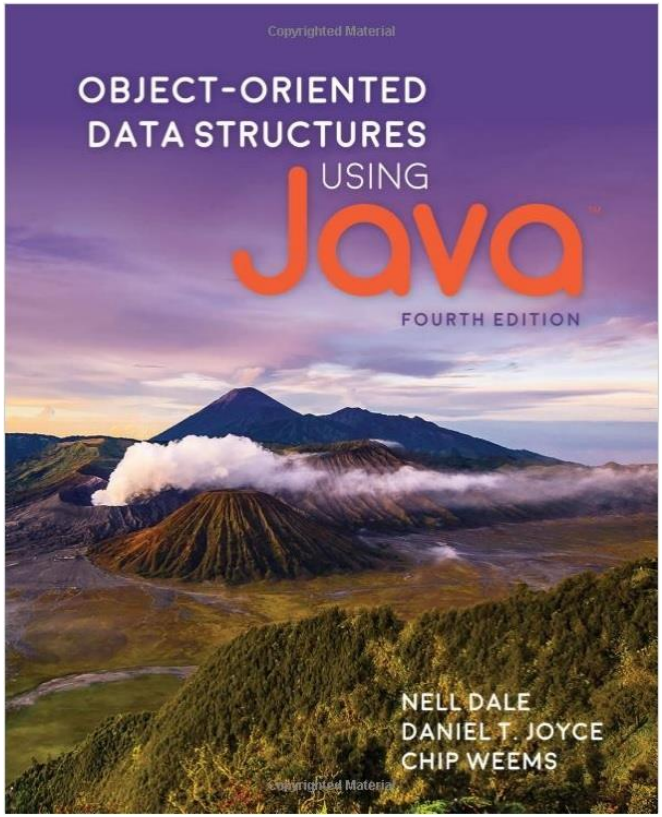
Text Book

"Object-Oriented Data Structures Using Java",

Nell Dale, Jones & Bartlett Learning; 4th edition (September 12, 2016).

ISBN-10: 1284089096.

ISBN-13: 978-1284089097.



Dr. Jonathan (Yahya) Cazalas

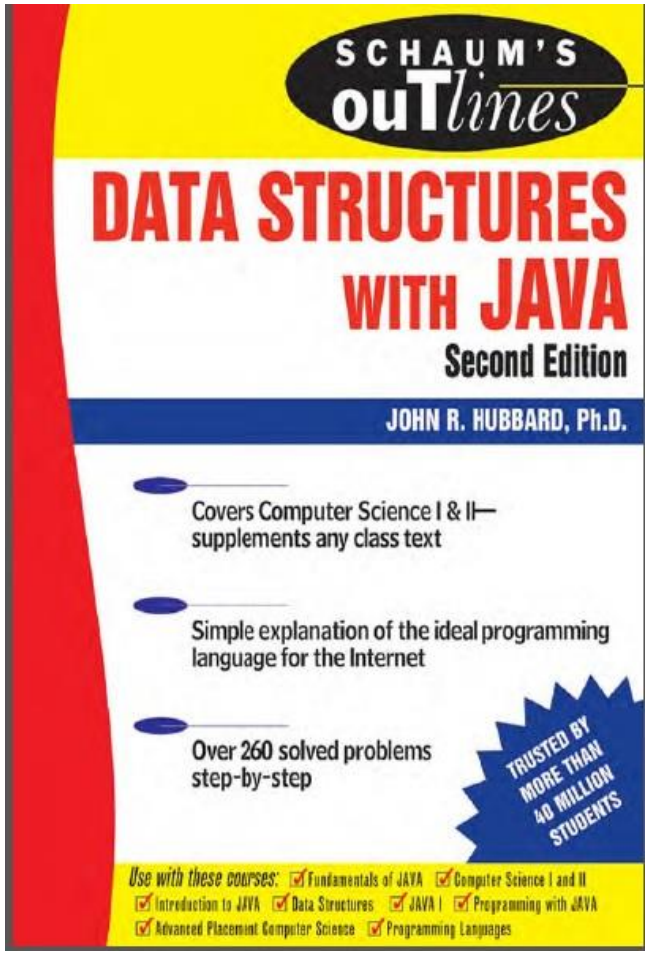
Dr. Muhammad Umair Ramzan

CPCS 204

Reference Book

"Schaum's Outlines, Data Structures with Java",

John R. Hubbard; McGraw-Hill; 2nd edition (2007). ISBN-07-147698-9.



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

General Policies

- Attendance must be 75% or more
- All written exams will be cumulative
- Quizzes will be on blackboard, No Makeup
- Plagiarism / cheating will result in 0% marks
- Pay regular visit to Blackboard
- Need help for Assignments; contact lab instructor
- **No scale up / bonus for final grade**

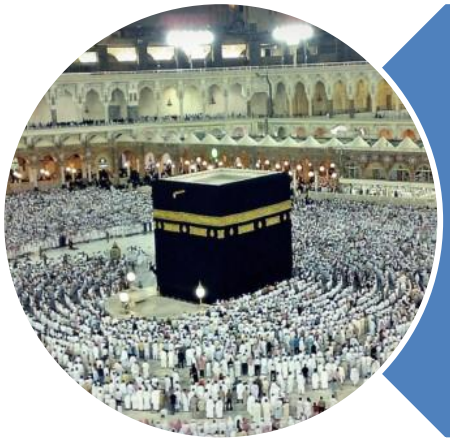
Assessments

Assessment	Grading Percentage
Quizzes	05%
Lab Work	15%
Assignments (3 * 5% each)	15%
Exam 1	15%
Exam 2	20%
Final Exam	30%

Data Structures - I

"No Bonus" marks for this course anymore

C P C S 2 0 4



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).(Quran 3 : 173)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan